



Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 5 · Day 3

Ensemble Learning Models: Decision Trees, Random Forests, Gradient Boosting, & XGBoost

Course Trainer: Victor Wandera Lumumba, MSc.

Role: Statistician, Data Analyst & ML Expert

Session Date: September 2026

Mode: Hybrid – Online / Physical (EAT)

“Ensemble methods transform weak learners into robust, clinical-grade predictors. Understanding the mathematics of bagging, boosting, and regularization is the key to unlocking state-of-the-art performance.”

About This Document

These comprehensive notes accompany **Week 5, Day 3** of the **Advanced Data Analytics & Machine Learning** program at Mediacrest Training College. Building upon the foundational classifiers explored in previous sessions, this module dives deep into **Ensemble Learning**, the cornerstone of modern tabular machine learning.

We will rigorously explore the mathematical foundations, algorithmic assumptions, and optimization techniques of Decision Trees, Random Forests, AdaBoost, Gradient Boosting Machines (GBM), and Extreme Gradient Boosting (XGBoost). Furthermore, we will bridge these advanced algorithms with rigorous clinical evaluation metrics (ROC-AUC, PR-AUC, F1-Score) using the ‘Heart.csv’ dataset, ensuring models are not only accurate but also safe and interpretable for healthcare deployment.

Course at a Glance

	8 Weeks (20+ intensive sessions)
Duration	
Core Tools	Python (pandas, NumPy, scikit-learn, XGBoost, matplotlib, seaborn)
Focus	Ensemble theory, bias-variance tradeoff, hyperparameter tuning, clinical evaluation
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Context	Healthcare, Clinical Risk Prediction, and Epidemiological Modeling

How to Use These Notes

- Master the Math:** The **Mathematical Foundation** boxes contain rigorous derivations essential for understanding *why* ensemble algorithms behave the way they do.
- Follow along in code:** Every listing is runnable. Open the companion Jupyter Notebook and execute each cell as you read.
- Contextualize:** **Kenyan Clinical Context** boxes connect abstract math to local healthcare realities (e.g., MOH policy, resource constraints, false negative costs).

Contents

About This Document	1
1 Foundations of Ensemble Learning	3
1.1 The Bias-Variance Tradeoff	3
1.2 The Heart Disease Dataset ('Heart.csv')	3
2 Decision Trees: The Building Blocks	3
2.1 Classification and Regression Trees (CART)	3
2.2 Splitting Criteria: Information Gain	4
2.3 Cost-Complexity Pruning	4
3 Bagging and Random Forests	4
3.1 Bootstrap Aggregating (Bagging)	4
3.2 Random Forests: Decorrelating the Trees	5
3.3 Feature Importance in Random Forests	5
4 Boosting Algorithms	5
4.1 AdaBoost (Adaptive Boosting)	5
4.2 Gradient Boosting Machines (GBM)	6
5 Extreme Gradient Boosting (XGBoost)	7
5.1 Regularized Objective Function	7
5.2 Second-Order Taylor Approximation	7
6 Model Evaluation & Clinical Trade-offs	8
6.1 Beyond Accuracy: The Confusion Matrix	8
6.2 ROC Curve and AUC	8
6.3 Precision-Recall (PR) Curve	8
7 Practical Implementation on 'Heart.csv'	9
7.1 Unified Preprocessing Pipeline	9
7.2 Model Definition & Hyperparameter Tuning	10
7.3 Hold-Out Test Set Evaluation	10
8 Summary & Policy Recommendations	11
8.1 Algorithm Selection Guide for Clinical Data	11
9 Glossary of Key Mathematical Terms	11
10 References & Further Reading	12

1 Foundations of Ensemble Learning

1.1 The Bias-Variance Tradeoff

The fundamental motivation for ensemble learning lies in the decomposition of a model's expected generalization error. For a regression problem (which conceptually extends to classification via probability estimation), the expected prediction error at a point x can be decomposed into three components:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2(\hat{f}(x)) + \text{Variance}(\hat{f}(x)) + \text{Irreducible Error}$$

- **Bias:** The error introduced by approximating a real-world problem with a simplified model. High-bias models (e.g., shallow decision trees, linear regression) underfit.
- **Variance:** The error introduced by the model's sensitivity to small fluctuations in the training set. High-variance models (e.g., deep, unpruned decision trees) overfit.

The Ensemble Solution

Ensemble methods combine multiple base estimators to improve generalizability and robustness over a single estimator.

- **Bagging (Bootstrap Aggregating):** Reduces **variance** by training independent models on bootstrapped datasets and averaging their predictions.
- **Boosting:** Reduces **bias** (and variance) by training models sequentially, where each new model focuses on the errors made by the previous ones.

1.2 The Heart Disease Dataset ('Heart.csv')

This dataset contains clinical cardiovascular measurements for 303 patients.

- **Target Variable:** HD (0 = No Heart Disease, 1 = Heart Disease).
- **Numeric Features:** Age, RestBP, Chol, MaxHR, Oldpeak.
- **Categorical/Ordinal Features:** Sex, ChestPain, Fbs, RestECG, ExAng, Slope, Ca, Thal.

Kenyan Clinical Context

In Kenyan county hospitals, non-communicable diseases (NCDs) like cardiovascular disease are rising. A robust classification model can act as a triage tool, helping Community Health Promoters (CHPs) identify high-risk individuals for early referral to facilities like Kenyatta National Hospital (KNH), optimizing limited specialist resources.

2 Decision Trees: The Building Blocks

2.1 Classification and Regression Trees (CART)

Decision trees partition the feature space $\mathcal{X}$ into M distinct, non-overlapping regions $R_1, R_2, \dots, R_M$. For classification, the prediction in region R_m is the majority class. The tree is grown by recur-

sively splitting nodes to maximize the homogeneity (purity) of the resulting child nodes.

Impurity Measures

Let p_{mk} be the proportion of class k observations in node m . Two common impurity measures are:

1. **Gini Impurity:** Measures the probability of misclassifying a randomly chosen element.

$$Gini(p_m) = 1 - \sum_{k=1}^K p_{mk}^2$$

2. **Entropy (Information Theory):** Measures the amount of disorder or uncertainty.

$$H(p_m) = - \sum_{k=1}^K p_{mk} \log_2(p_{mk})$$

2.2 Splitting Criteria: Information Gain

At each node, the algorithm evaluates all possible splits across all features. The quality of a split is measured by the reduction in impurity, known as **Information Gain** (or Gini Gain).

$$\text{Gain}(D, A) = \text{Impurity}(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \text{Impurity}(D_v)$$

where D is the dataset at the current node, A is the feature being tested, and D_v is the subset of data where feature A takes value v . The split that maximizes this gain is selected.

2.3 Cost-Complexity Pruning

Unpruned trees tend to overfit the training data. **Cost-Complexity Pruning** (also known as weakest link pruning) introduces a regularization term to penalize tree size.

$$R_\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ is the training error (e.g., misclassification rate), $|T|$ is the number of terminal nodes (leaves), and $\alpha \geq 0$ is the complexity parameter. As α increases, the penalty for complexity increases, leading to smaller, more generalized trees. The optimal α is typically chosen via cross-validation.

3 Bagging and Random Forests

3.1 Bootstrap Aggregating (Bagging)

Bagging reduces the variance of high-variance base learners (like deep decision trees).

1. Draw B bootstrap samples (samples with replacement) from the original training dataset D .
2. Train a separate decision tree T_b on each bootstrap sample D_b .

3. For a new observation x , aggregate the predictions:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (\text{Regression})$$

$$\hat{f}_{bag}(x) = \text{majority_vote}\{T_1(x), \dots, T_B(x)\} \quad (\text{Classification})$$

Out-of-Bag (OOB) Error Estimation

Each bootstrap sample contains approximately 63.2% of the original data. The remaining 36.8% are "Out-of-Bag" (OOB) observations. These can be used as a validation set to estimate the generalization error without needing a separate hold-out set:

$$\text{OOB Error} = \frac{1}{N} \sum_{i=1}^N I(y_i \neq \hat{y}_i^{OOB})$$

where $\hat{y}_i^{OOB}$ is the aggregated prediction from all trees that did *not* include observation i in their bootstrap sample.

3.2 Random Forests: Decorrelating the Trees

A limitation of bagging is that if one feature is very strong, most trees will use it for their first split, making the trees highly correlated. Averaging highly correlated trees does not reduce variance as much as averaging independent trees.

Random Forests introduce an additional layer of randomness: at each split, the algorithm considers only a random subset of m features (typically $m \approx \sqrt{p}$ for classification, where p is the total number of features). This **Random Subspace Method** forces the trees to explore different features, decorrelating them and significantly reducing variance.

3.3 Feature Importance in Random Forests

Random Forests provide a built-in measure of feature importance, known as **Mean Decrease in Impurity** (MDI). For a feature A , its importance is the total reduction in node impurity (weighted by the probability of reaching that node) averaged over all trees in the forest:

$$\text{Importance}(A) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in \text{Tree}_b} \Delta \text{Impurity}(t) \cdot P(t)$$

where $P(t)$ is the proportion of samples reaching node t .

4 Boosting Algorithms

4.1 AdaBoost (Adaptive Boosting)

AdaBoost builds an ensemble of weak learners (typically decision stumps, i.e., trees with depth 1) sequentially. It assigns higher weights to misclassified observations, forcing subsequent models to focus on the "hard" examples.

AdaBoost Mathematical Formulation

1. **Initialize** observation weights: $w_i^{(1)} = \frac{1}{N}$ for $i = 1, \dots, N$.
2. **For** $m = 1$ to M (number of estimators):
 - (a) Fit a weak classifier $G_m(x)$ to the training data using weights $w^{(m)}$.
 - (b) Compute the weighted error rate: $err_m = \frac{\sum_{i=1}^N w_i^{(m)} I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i^{(m)}}$.
 - (c) Compute the model weight (importance): $\alpha_m = \frac{1}{2} \ln \left(\frac{1 - err_m}{err_m} \right)$.
 - (d) Update the observation weights: $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i G_m(x_i))$.
 - (e) Normalize weights so they sum to 1.
3. **Final Model:** $F(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$.

4.2 Gradient Boosting Machines (GBM)

While AdaBoost adjusts observation weights, Gradient Boosting fits new models to the **residual errors** (or negative gradients) of the previous models. It performs gradient descent in function space.

Forward Stagewise Additive Modeling

We wish to find a function $F(x)$ that minimizes the expected loss $\mathbb{E}[L(y, F(x))]$. We build the model additively:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

where $h_m(x)$ is a base learner (e.g., a decision tree) and γ_m is the step size.

1. **Initialize:** $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.
2. **For** $m = 1$ to M :
 - (a) Compute the **pseudo-residuals** (negative gradient of the loss):

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

- (b) Fit a base learner $h_m(x)$ to the pseudo-residuals r_{im} .
- (c) Compute the optimal step size γ_m via line search:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$$

- (d) Update the model: $F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$.

5 Extreme Gradient Boosting (XGBoost)

XGBoost is an optimized, scalable implementation of Gradient Boosting. It introduces several key innovations, most notably a **regularized objective function** and a **second-order Taylor approximation** of the loss function, which leads to faster convergence and better generalization.

5.1 Regularized Objective Function

The objective function to be minimized at iteration t is:

$$Obj^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k)$$

where $\Omega(f)$ is the regularization term penalizing the complexity of the tree f :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

Here, T is the number of leaves in the tree, w is the vector of leaf weights, γ controls the minimum loss reduction required to make a further partition (pruning), and λ is the L2 regularization term on the leaf weights (preventing overfitting).

5.2 Second-Order Taylor Approximation

To efficiently optimize the objective, XGBoost approximates the loss function L using a second-order Taylor expansion around the current prediction $\hat{y}_i^{(t-1)}$:

$$Obj^{(t)} \approx \sum_{i=1}^n \left[L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

where $g_i = \partial_{\hat{y}^{(t-1)}} L(y_i, \hat{y}^{(t-1)})$ is the first derivative (gradient) and $h_i = \partial_{\hat{y}^{(t-1)}}^2 L(y_i, \hat{y}^{(t-1)})$ is the second derivative (Hessian).

Optimal Leaf Weights and Split Gain

By defining $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$ as the sum of gradients and Hessians in leaf j , we can solve for the optimal weight w_j^* for each leaf by setting the derivative to zero:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Substituting this back into the objective gives the optimal structure score for a given tree structure q :

$$Obj^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

The **Gain** of a split is the difference between the score of the new tree (with left and right children) and the score of the original tree (before the split):

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{G_L + G_R + \lambda} \right] - \gamma$$

A split is only accepted if the $\text{Gain} > 0$.

6 Model Evaluation & Clinical Trade-offs

6.1 Beyond Accuracy: The Confusion Matrix

In clinical datasets, accuracy is dangerously misleading due to class imbalance and asymmetric error costs. We must decompose predictions into four quadrants:

- **True Positive (TP):** Model predicts Disease, Patient has Disease.
- **False Positive (FP):** Model predicts Disease, Patient is Healthy. (Cost: Unnecessary, expensive angiogram, patient anxiety).
- **False Negative (FN):** Model predicts Healthy, Patient has Disease. (Cost: Missed diagnosis, potential mortality).
- **True Negative (TN):** Model predicts Healthy, Patient is Healthy.

Derived Clinical Metrics

$$\begin{aligned}\text{Precision (PPV)} &= \frac{TP}{TP + FP} \quad (\text{Of all flagged, how many are truly sick?}) \\ \text{Recall (Sensitivity)} &= \frac{TP}{TP + FN} \quad (\text{Of all truly sick, how many did we catch?}) \\ \text{Specificity} &= \frac{TN}{TN + FP} \quad (\text{Of all healthy, how many were correctly cleared?}) \\ \text{F1-Score} &= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{Harmonic mean, balances both}) \\ \text{False Negative Rate (FNR)} &= \frac{FN}{FN + TP} = 1 - \text{Recall}\end{aligned}$$

6.2 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate ($1 - \text{Specificity}$) at various classification thresholds.

$$\text{FPR} = \frac{FP}{FP + TN}$$

The **Area Under the Curve (AUC)** represents the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative instance. An AUC of 0.5 is random guessing; 1.0 is perfect.

6.3 Precision-Recall (PR) Curve

For imbalanced datasets (or when the positive class is of primary interest), the PR curve is more informative than the ROC curve. It plots Precision against Recall. The **Average Precision (AP)** or PR-AUC summarizes this curve. A high PR-AUC indicates that the model maintains high precision even as recall increases.

Threshold Selection for Policy

- **Primary Care Screening (CHPs):** Prioritize **Recall**. Lower the threshold (e.g., 0.3). We accept more FPs to ensure no sick patient is missed (FNs are catastrophic).
- **Specialist Referral (Cardiology):** Prioritize **Precision**. Raise the threshold (e.g., 0.7). We must be highly confident before recommending an invasive, costly procedure at a referral hospital.

7 Practical Implementation on ‘Heart.csv’

7.1 Unified Preprocessing Pipeline

We must handle missing values, encode categoricals, and scale numericals. Tree-based models are generally robust to unscaled data, but scaling is included for completeness and for any linear components.

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split, RandomizedSearchCV,
   StratifiedKFold
4 from sklearn.compose import ColumnTransformer
5 from sklearn.preprocessing import StandardScaler, OrdinalEncoder
6 from sklearn.impute import SimpleImputer
7 from sklearn.pipeline import Pipeline
8
9 # 1. Load Data
10 df = pd.read_csv('Heart.csv', na_values=['NA', '?', 'NaN', ''])
11
12 # 2. Imputation
13 df['Ca'] = pd.to_numeric(df['Ca'], errors='coerce').fillna(df['Ca'].median())
14 df['Thal'] = df['Thal'].fillna('unknown')
15
16 # 3. Define Features and Target
17 X = df.drop(columns=['HD', 'AHD'])
18 y = df['HD']
19
20 # 4. Preprocessor
21 numeric_features = ['Age', 'RestBP', 'Chol', 'MaxHR', 'Oldpeak']
22 categorical_features = ['Sex', 'ChestPain', 'Fbs', 'RestECG', 'ExAng', 'Slope',
   'Ca', 'Thal']
23
24 preprocessor = ColumnTransformer(
25     transformers=[
26         ('num', SimpleImputer(strategy='median'), numeric_features),
27         ('cat', SimpleImputer(strategy='most_frequent'), categorical_features)
28     ])
29
30 # 5. Train/Test Split (Stratified to maintain class balance)
31 X_train, X_test, y_train, y_test = train_test_split(
32     X, y, test_size=0.2, random_state=42, stratify=y
33 )

```

Listing 1: Robust Preprocessing Pipeline

7.2 Model Definition & Hyperparameter Tuning

We use `RandomizedSearchCV` with 5-fold stratified cross-validation to efficiently explore the hyperparameter space, optimizing for `roc_auc`.

```

1 from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier,
  GradientBoostingClassifier, AdaBoostClassifier
2 import xgboost as xgb
3 from sklearn.metrics import roc_auc_score, classification_report,
  confusion_matrix
4
5 # Define pipelines
6 rf_pipe = Pipeline([('prep', preprocessor), ('rf', RandomForestClassifier(
  random_state=42, class_weight='balanced'))])
7 et_pipe = Pipeline([('prep', preprocessor), ('et', ExtraTreesClassifier(
  random_state=42, class_weight='balanced'))])
8 gb_pipe = Pipeline([('prep', preprocessor), ('gb', GradientBoostingClassifier(
  random_state=42))])
9 xgb_pipe = Pipeline([('prep', preprocessor), ('xgb', xgb.XGBClassifier(
  random_state=42, eval_metric='logloss', scale_pos_weight=(y_train==0).sum()
  /(y_train==1).sum()))])
10
11 # Example Parameter Grid for Random Forest
12 param_dist_rf = {
13     'rf__n_estimators': [100, 200, 300, 500],
14     'rf__max_depth': [None, 10, 20, 30],
15     'rf__min_samples_split': [2, 5, 10],
16     'rf__min_samples_leaf': [1, 2, 4]
17 }
18
19 # Execute RandomizedSearchCV
20 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
21 search_rf = RandomizedSearchCV(rf_pipe, param_dist_rf, n_iter=30, cv=cv, scoring
  ='roc_auc', n_jobs=-1, random_state=42)
22 search_rf.fit(X_train, y_train)
23
24 print(f"Best RF ROC-AUC: {search_rf.best_score_:.4f}")
25 best_rf_model = search_rf.best_estimator_

```

Listing 2: Ensemble Model Pipelines and Tuning

7.3 Hold-Out Test Set Evaluation

After selecting the best model via cross-validation, we evaluate it on the completely unseen hold-out test set to estimate real-world performance.

```

1 # Predictions
2 y_pred = best_rf_model.predict(X_test)
3 y_prob = best_rf_model.predict_proba(X_test)[:, 1]
4
5 # Metrics
6 cm = confusion_matrix(y_test, y_pred)
7 tn, fp, fn, tp = cm.ravel()
8
9 print("--- Random Forest Test Set Performance ---")
10 print(f"Accuracy: { (tp+tn)/(tp+tn+fp+fn):.4f}")

```

```

11 print(f"Precision: { tp/(tp+fp):.4f}")
12 print(f"Recall:      { tp/(tp+fn):.4f}")
13 print(f"F1-Score:    { 2*tp/(2*tp+fp+fn):.4f}")
14 print(f"ROC-AUC:     { roc_auc_score(y_test, y_prob):.4f}")
15 print(f"FNR:        { fn/(fn+tp):.4f}")
16 print("\nConfusion Matrix:")
17 print(f"TN={tn:2d} | FP={fp:2d}")
18 print(f"FN={fn:2d} | TP={tp:2d}")

```

Listing 3: Comprehensive Test Set Evaluation

8 Summary & Policy Recommendations

8.1 Algorithm Selection Guide for Clinical Data

- **Random Forest:** The best **overall choice** for tabular clinical data. It provides an excellent balance of high accuracy, robustness to overfitting, and built-in feature importance, while requiring minimal hyperparameter tuning.
- **Extra Trees:** The best **sensitivity-first alternative**. By using random splits, it introduces more randomness, often resulting in higher Recall (lower False Negative Rate), which is critical when missing a disease is catastrophic.
- **XGBoost:** The best choice for **maximum predictive performance** on large datasets. Its regularized objective function prevents overfitting, but it requires careful hyperparameter tuning and is less interpretable than Random Forest.
- **Gradient Boosting (GBM):** A strong baseline for boosting, but generally outperformed by XGBoost in both speed and accuracy due to the lack of second-order optimization and built-in regularization.

Capstone Preparation: The Algorithmic Audit

Imagine you are deploying the Extra Trees model to a county hospital. The hospital administrator asks: *"Why did the model flag this 45-year-old female patient as high risk, and why should we trust it over a simple rule?"*

1. Explain the trade-off made: Extra Trees was chosen specifically to minimize False Negatives (Recall = 96.43%), accepting a slightly higher False Positive rate to ensure no sick patient is missed.
2. Propose a solution using Explainable AI (XAI) techniques (e.g., SHAP values) to show exactly which features (e.g., ChestPain type, Oldpeak) drove this specific prediction.
3. Draft a 2-sentence response to the administrator that balances technical honesty with clinical reassurance.

9 Glossary of Key Mathematical Terms

Bootstrap Aggregating A technique to reduce variance by training multiple models on different bootstrap samples and averaging their predictions.

Cost-Complexity Pruning	A regularization technique for decision trees that penalizes the number of leaves to prevent overfitting.
Gini Impurity	A measure of how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset.
Hinge Loss	The loss function used in SVMs, though conceptually related to the margin-maximization goal of some boosting variants.
Pseudo-residuals	The negative gradient of the loss function with respect to the model's predictions, used as the target variable for the next weak learner in Gradient Boosting.
Taylor Expansion	A mathematical approximation of a function as an infinite sum of terms calculated from the values of its derivatives at a single point; used in XGBoost to approximate the loss function.

10 References & Further Reading

- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
- Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 29(5), 1189-1232.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. (Chapters 9 & 10 for Trees and Ensemble Methods).
- Mediacrest Training College. (2026). *Advanced Data Analytics & Machine Learning Curriculum*.

Mediacrest Training College – Nairobi, Kenya

Advanced Data Analytics & Machine Learning · Week 5, Day 3

Trainer: Victor Wandera Lumumba, MSc. · September 2026
